Файлы CSV

Теория

```
date;film;price;sold
01.10;Лето;350,5;42
02.10;"Он; она";280;9
03.10;Свет;300;31
```



Только то, что нужно к задаче на таблицу на школьном этапе. Пример один — файл `films.csv`. Код идёт подряд, как ячейки ноутбука: что объявлено выше, тем пользуются ниже. Под кодом, с чертой слева, — ровно то, что он печатает.

Что такое CSV

Таблица, записанная текстом: строка файла — строка таблицы, значения через запятую, первая строка — названия столбцов. Файл `films.csv`:

```
title,genre,year,rating,views
Северный ветер,драма,1998,7.4,120
Город,комедия,2004,6.9,340
Лето, Драма,2011,8.0,95
Мост,триллер,2004,7.1,210
Остров,драма,2006,,60
Свет,комедия,2015,н/д,410
Река,триллер,2019,7.8,150
Сад,драма,2021,6.5,80
```

В нём две неаккуратности, обычные для олимпиадных таблиц: у «Острова» оценки нет, у «Света» вместо неё `н/д`, а у «Лета» жанр с пробелом и заглавной буквой.

Чтение

```
import pandas as pd
df = pd.read_csv("films.csv")
print(df)
print(df.shape)
```

```
             title    genre  year rating  views
0  Северный ветер    драма  1998    7.4    120
1           Город  комедия  2004    6.9    340
2            Лето    Драма  2011    8.0     95
3            Мост  триллер  2004    7.1    210
4          Остров    драма  2006    NaN     60
5            Свет  комедия  2015    н/д    410
6            Река  триллер  2019    7.8    150
7             Сад    драма  2021    6.5     80
(8, 5)
```

Слева — номера строк, `df.shape` — сколько строк и столбцов. Длинную таблицу целиком не печатают: `df.head()` — первые пять строк.

Если файл прочитался не так:

- всё склеилось в один столбец — разделитель не запятая: `pd.read_csv("f.csv", sep=";")`;
- числа записаны с запятой, `350,5`, — добавить `decimal=","`;
- ошибка `UnicodeDecodeError` — другая кодировка: `encoding="cp1251"`.

Чистка

Пустое поле pandas читает как пропуск `NaN`, а `н/д` — как текст, и тогда текстом становится весь столбец: числа в нём уже не посчитать. Лишний пробел и заглавная буква делают из драмы ещё один жанр:

```
print(df["rating"].tolist())
print(df["genre"].value_counts())
```

```
['7.4', '6.9', '8.0', '7.1', nan, 'н/д', '7.8', '6.5']
genre
драма      3
комедия    2
триллер    2
 Драма     1
Name: count, dtype: int64
```

Чинится двумя строками:

```
df["rating"] = pd.to_numeric(df["rating"],
                             errors="coerce")
df["genre"] = df["genre"].str.strip().str.lower()
print(df["rating"].tolist())
print(df["genre"].value_counts())
```

```
[7.4, 6.9, 8.0, 7.1, nan, nan, 7.8, 6.5]
genre
драма      4
комедия    2
триллер    2
Name: count, dtype: int64
```

`to_numeric(..., errors="coerce")` делает пропуском всё, что не число. `str.strip()` убирает пробелы по краям, `str.lower()` переводит в строчные. Что ещё бывает нужно:

- `df["rating"].isna().sum()` — сколько пропусков;

- `df = df.dropna(subset=["rating"])` — выбросить строки с пропуском;
- `df["rating"] = df["rating"].fillna(0)` — заполнить пропуски;
- `df = df.drop_duplicates()` — убрать повторы строк.

Такие команды таблицу не меняют, а возвращают новую — поэтому результат присваивают обратно.

Выбор строк

Условие — в квадратных скобках после таблицы. Каждое сравнение — в круглых, «и» — `&`, «или» — `|`, «не» — `~`:

```
d = df[(df["genre"] == "драма")
       & (df["year"] >= 2000)]
print(d)
print(len(d))
```

```
    title  genre  year  rating  views
2    Лето  драма  2011     8.0     95
4  Остров  драма  2006     NaN     60
7     Сад  драма  2021     6.5     80
3
```

`len(d)` — сколько строк подошло. Ещё:

- `df[df["year"].between(2000, 2010)]` — оба конца входят; если один не входит — двумя сравнениями;
- `df[df["genre"].isin(["драма", "триллер"])]` — одно из значений;
- `df[df["rating"].isna()]` — строки с пропуском.

Счёт по столбцу

```
v = df["views"]
print(v.sum(), v.mean(), v.median(), v.max())
print(df["rating"].mean(), df["rating"].count())
print(df.loc[v.idxmax(), "title"])
```

```
1465 183.125 135.0 410
7.283333333333334 6
Свет
```

Средняя оценка — по шести фильмам, а не по восьми: пропуски не считаются, `count()` их тоже не включает. `idxmax()` — номер строки с наибольшим значением, `df.loc[номер, "столбец"]` — что стоит в этой строке. Ещё `min()` и `idxmin()`, `nunique()` — сколько различных значений, `mode()[0]` — самое частое.

Новый столбец считают формулой из старых, сразу по всем строкам:

```
df["age"] = 2026 - df["year"]
print(df[["title", "year", "age"]].head(3))
```

```
             title  year  age
0  Северный ветер  1998   28
1           Город  2004   22
2            Лето  2011   15
```

Группы и сортировка

`groupby` — то же, что сводная таблица в Excel: строки делятся по значению столбца, и счёт идёт по каждой группе отдельно.

```
g = df.groupby("genre")["views"].sum()
print(g)
print(g.idxmax(), g.max())
print(df.groupby("genre")["rating"].mean().round(2))
```

```
genre
драма      355
комедия    750
триллер    360
Name: views, dtype: int64
комедия 750
genre
драма      7.30
комедия    6.90
триллер    7.45
Name: rating, dtype: float64
```

Вместо `sum` подходит всё из счёта по столбцу; `df.groupby("genre").size()` — сколько строк в каждой группе. Результат — снова столбец, и выбор строк к нему применяется так же: `g[g > 500]` — группы, где больше 500.

```
top = df.sort_values("views", ascending=False)
print(top[["title", "views"]].head(3))
```

```
   title  views
5   Свет    410
1  Город    340
3   Мост    210
```

`ascending=False` — по убыванию, без него — по возрастанию.

Ответ

Округлять — самому. `round` в Python округляет половину к чётному, а не вверх, как в школе:

```
print(round(2.5), round(3.5), round(0.125, 2))
print(f"{df['rating'].mean():.4f}")
```

```
2 4 0.12
7.2833
```

Поэтому число печатают с запасом знаков, как во второй строке, и округляют по условию руками.

Ответ файлом — без номеров строк, `index=False`:

```
res = g.reset_index()
res.to_csv("answer.csv", index=False)
print(open("answer.csv", encoding="utf-8").read())
```

```
genre,views
драма,355
комедия,750
триллер,360
```

`reset_index()` превращает результат группировки обратно в таблицу из двух столбцов. Названия и порядок столбцов — как в условии.